Manual de despliegue - Fase II

Objetivo

Explicar cómo reconstruir, ejecutar y validar una aplicación de riesgo crediticio en un contenedor Docker **sin instalar software en Windows 11**, usando un equipo remoto de GitHub Codespaces. También incluye el procedimiento local para evaluadores que dispongan de Docker.

Alcance y requisitos

- Cuenta de GitHub con acceso a Codespaces y navegador web.
- Cuota de cómputo suficiente; revisar los límites de uso de la cuenta antes de iniciar.
- Acceso público al dataset de Kaggle (`uciml/default-of-credit-card-clients-dataset`).
- No se requieren credenciales de Kaggle para este dataset público en condiciones normales, aunque el proveedor puede cambiar restricciones.
- No se requiere GPU, WSL, Docker Desktop ni instalar Python en Windows.

A. Importación desde Windows

1. Extraer el ZIP con **Extraer todo** de Windows.
2. Crear repositorio en https://github.com/new (nombre sugerido `riesgo-crediticio-fase-ii`; privado recomendado).
3. En la interfaz web del repositorio, usar **Add file > Upload files** para subir el contenido extraído, manteniendo rutas, incluida la carpeta `.devcontainer` y `.github`. Hacer commit.
4. Comprobar que `Dockerfile`, `app/`, `scripts/`, `tests/` y `README.md` aparecen en el repositorio.

B. Preparar Codespaces

1. En el repositorio: **Code > Codespaces > Create codespace on main**.
2. Confirmar que se abre una ventana de desarrollo en el navegador. El archivo `.devcontainer/devcontainer.json` habilita Docker-in-Docker.
3. En su terminal ejecutar `docker version` y `python --version` para confirmar el entorno.
4. Si el comando `docker` no está disponible o el daemon no responde, seleccionar **Rebuild Container** de Codespaces; no intentar instalar Docker en Windows.

C. Guardar el modelo una vez y arrancar Docker sin reentrenar

Primera vez (solo si no existen los artefactos en el repositorio):

```
bash scripts/prepare_model.sh
```

El comando instala las dependencias de entrenamiento, verifica el CSV de Kaggle, reproduce el experimento, exporta `artifacts/modelo_riesgo.joblib` y `artifacts/metadata.json`, y comprueba el SHA256. Comprueba que el archivo `.joblib` pese menos de 100 MB. Para conservar la misma versión entre Codespaces distintos, guardar esos archivos en GitHub:

```
git add artifacts/modelo_riesgo.joblib artifacts/metadata.json
git commit -m "Guardar modelo entrenado de riesgo crediticio"
git push
```

Cada inicio de sesión o después de un reinicio (sin entrenar):

```
bash scripts/start_codespaces.sh
```

El script comprueba los artefactos mediante biblioteca estándar de Python, inicia un contenedor existente o lo recrea a partir del modelo guardado. Si cambian los archivos de la app o del modelo, usa `bash scripts/start_codespaces.sh --rebuild`. Si faltan artefactos, detiene el arranque con instrucciones claras **sin entrenar automáticamente**. El modelo se carga una vez por proceso y luego sirve muchas solicitudes.

Codespaces conserva `/workspaces` entre paradas/reconstrucciones del mismo entorno, pero las imágenes y contenedores pueden perderse; hacer commit y push asegura los artefactos al clonar otro Codespace. El contenedor no es un servicio público permanente: deja de atender cuando Codespaces se detiene.

D. Verificar el servicio desde el navegador

1. En Codespaces, abrir **PORTS** y localizar el puerto 8000.
2. Mantener la visibilidad **Private**, seleccionar la URL del puerto 8000 y abrir el formulario.
3. Probar el ejemplo predeterminado sin introducir datos personales.
4. Abrir `/docs` para ver OpenAPI y los endpoints.
5. En la terminal de Codespaces ejecutar `bash scripts/smoke_docker.sh`.
6. Ejecutar `docker ps` y `docker logs riesgo-crediticio-demo` para revisar la ejecución del contenedor.
7. Capturar las pantallas auténticas para la evidencia académica: consola de construcción, `docker ps`, `/api/health`, UI, `/docs`, pruebas HTTP.

E. Pruebas independientes en GitHub Actions

1. En el repositorio, pestaña **Actions**. Si GitHub solicita habilitar workflows, revisar y autorizar.
2. El workflow `Pruebas de API (sin dataset externo)` se ejecuta en cada push: valida cinco pruebas unitarias sin modelos reales.
3. Ejecutar manualmente `Construir y verificar Docker con modelo versionado` desde **Run workflow**. El workflow comprueba la integridad del modelo versionado, construye la imagen Docker y verifica la API sin reentrenar.
4. Inspeccionar las salidas reales de cada paso; no afirmar éxito hasta ver el indicador verde en GitHub.

F. Ruta alternativa de Docker local (para el evaluador)

En cualquier equipo con Docker Engine disponible y los artefactos exportados (o clonados desde GitHub):

```
docker build -t riesgo-crediticio-faseii:1.0 .
docker run --rm -p 8000:8000 riesgo-crediticio-faseii:1.0
```

Después acceder a `http://localhost:8000/` y `http://localhost:8000/docs`. Si faltan los artefactos, ejecutar primero `bash scripts/prepare_model.sh` en el entorno de desarrollo y realizar commit/push para no repetir el entrenamiento.

G. Manual orientativo para un PaaS con despliegue Docker

Se puede migrar a un proveedor PaaS que admita `Dockerfile` y enlazar el repositorio. El servicio debe disponer del `modelo_riesgo.joblib` generado de antemano, ya sea incorporado en una imagen construida en un pipeline confiable o mediante un almacén privado de artefactos; el archivo de entrega inicial no contiene ese binario; se guarda posteriormente en GitHub junto a metadata.json. Configurar puerto 8000, ruta de salud `/api/health`, límites de CPU/RAM y gestión de logs. Revisar las tarifas vigentes antes de desplegar. No poner esta demostración en acceso público sin autenticación, controles de abuso y revisión de seguridad.

Importante: Codespaces es un entorno de desarrollo remoto, no un servicio PaaS de producción. Este manual documenta la ruta al despliegue, pero no afirma que ya exista un sitio público.

H. Diagnóstico rápido

- Error al descargar Kaggle: revisar conectividad / restricciones de acceso / dataset original.
- Hash SHA256 distinto: no continuar con un CSV modificado; descargar el original.
- Faltan artefactos: ejecutar una sola vez `bash scripts/prepare_model.sh` y registrar artefactos en GitHub.
- `No module named joblib`: el entrenamiento se intento ejecutar sin preparar dependencias; usar `prepare_model.sh`. El inicio normal no necesita instalar librerias de ML fuera de Docker.
- `Connection refused`: confirmar `docker ps` y volver a ejecutar `bash scripts/start_codespaces.sh`.
- `docker start` muestra `No such container`: es normal si el daemon perdio contenedores; el script de arranque los recrea desde la imagen o archivos versionados.
- `docker build` falla en `COPY artifacts`: preparar modelo antes de reconstruir.
- `503` desde `/api/health`: revisar existencia e integridad de `modelo_riesgo.joblib` y `metadata.json`.
- `422` desde `/api/predecir`: enviar las 19 variables con tipos y rangos correctos.
- Codespaces no muestra localhost: acceder por el vínculo de la pestaña PORTS, no por localhost de Windows.
- La cuota se agota: detener Codespaces y revisar cuota de GitHub; no se garantiza uso gratuito ilimitado.

I. Seguridad y evidencia

Nunca usar datos personales ni subir tokens de GitHub/Kaggle. El puerto debe permanecer privado. Guardar evidencia de un contenedor realmente ejecutado, y evitar capturas de interfaz simuladas. Las pruebas unitarias con `FakeModel` no demuestran precisión ni entrenamiento del modelo real.

Fuentes

- Notebook original entregado con esta actividad.
- https://docs.github.com/en/codespaces
- https://docs.github.com/en/codespaces/developing-in-a-codespace/forwarding-ports-in-your-codespace
- https://github.com/devcontainers/features/tree/main/src/docker-in-docker
- https://docs.github.com/en/actions